

# Bags, Vectors & Transformers

Large Language Models & Data Annotation · Day 4, Part 1

Denise J. Roth, PhD Candidate

Strategic Communication Group · Wageningen University & Research

# From BERT to ChatGPT

Yesterday we met two families of transformers. Today we go deep on the second one.

## Encoders (BERT)

- Read the whole sentence
- Built for **understanding**
- Classification, search

*Day 3's focus.*

## Decoders (GPT)

- Predict the next word
- Built for **generation**
- Scaled up = **LLMs**

**Today's focus.**

*Take a decoder, make it enormous, train it on much of the internet — and you get a **Large Language Model**.*

# What an LLM Actually Does: Predict the Next Word

At heart, an LLM does something almost absurdly simple: given some text, it predicts **the most likely next word** — over and over.

“The capital of the Netherlands is”



“**Amsterdam**” (next word)

Then it **appends** that word and predicts again. Repeat, and coherent paragraphs emerge — one word at a time. That is all “generation” is.

The surprise of the last few years: doing *just this*, at **massive scale**, produces something that can summarize, translate, answer questions, write code, and reason — none of which it was explicitly taught.

# The Magic Ingredient: Scale

Why is GPT-4 so different from the small models of a few years ago? Mostly **scale** — three things grew together, by orders of magnitude:

## Data

Much of the public internet — **trillions** of words

## Parameters

From millions (BERT) to **hundreds of billions**

## Compute

Thousands of GPUs, months of training, huge cost

As models grew, **new abilities appeared** that smaller models simply did not have — following instructions, few-shot learning, reasoning. This is why they are called *large* language models.

*The flip side: that scale is expensive, concentrated in a few companies, and has real costs — we return to this in Part 2.*

# From Word-Predictor to Assistant

A raw next-word predictor is not yet ChatGPT. Two extra steps turn it into a helpful **assistant**:

## Instruction tuning

Fine-tune on examples of **following instructions**, so it answers your request instead of just continuing your text.

## Human feedback (RLHF)

People rate responses; the model is nudged toward **helpful, honest, harmless** answers.

*You mostly interact with instruction-tuned models — but under the hood, it is still next-word prediction.*

# What Can LLMs Do? A Sampler

## Text tasks

- **Summarize** long documents
- **Translate** between languages
- **Extract** structured info from messy text
- **Rewrite**, simplify, paraphrase
- **Answer questions** about a text

## Research tasks

- **Classify** & code text (topic, sentiment, stance)
- **Annotate** data at scale — our focus
- Draft **code** for analysis
- **Brainstorm** & refine measures
- Help with **literature** synthesis

*A general-purpose text tool — but “general-purpose” also means “not automatically reliable”.  
Every use needs validation.*

# How You Steer Them: Prompting

You control an LLM through the **prompt** — the instructions and context you give it. Three patterns you will use constantly:

## Zero-shot

Just ask.

*“Classify this tweet as positive, negative, or neutral.”*

## Few-shot

Give a few **examples** first.

*Show 3 labeled tweets, then ask for the 4th.*

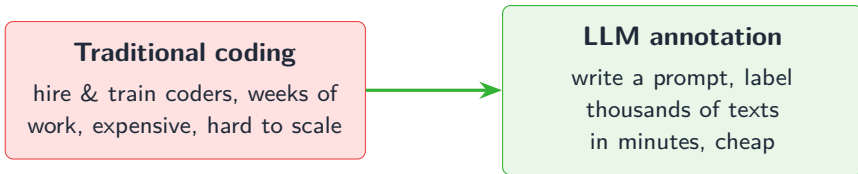
## Chain-of-thought

Ask it to **reason step by step** before answering — often more accurate.

*Prompt wording matters a lot — and that is both a strength (flexible) and a weakness (fragile, hard to reproduce). More on this in Part 2.*

# The Use Case We Care About: Data Annotation

For social scientists, one LLM use stands out: **annotating (coding) text data**. Traditionally this means armies of human coders. LLMs can do a lot of it — fast and cheap.



Give the model your **codebook** as a prompt, feed it each text, and collect the labels. Studies increasingly show LLMs can **match or approach human coders** on many tasks — at a fraction of the cost and time.

# How LLM Annotation Works in Practice

The recipe is simple. Turn your **codebook** into a prompt, loop over your texts, parse the answers.

```
You are an expert coder. Classify each text as one of:  
POSITIVE, NEGATIVE, or NEUTRAL toward the policy.
```

```
Reply with only the label.
```

```
Text: "The new bus plan is a total mess."
```

```
Label: ____
```

The same prompt runs across your whole corpus. You can add your **definitions**, **examples** (few-shot), and **edge-case rules** — exactly the things a human codebook contains.

# Annotation: Powerful, But Validate!

## The promise

- **Fast** & cheap at scale
- Consistent (no coder fatigue)
- Handles nuance better than dictionaries
- Easy to iterate on the codebook

## The catch

- Can be **confidently wrong**
- Sensitive to prompt wording
- May carry **bias**
- **Always validate** against human-coded gold standard

*The golden rule: hand-code a sample yourself, measure agreement with the LLM, and report it. An LLM annotator is a **measurement instrument** — and instruments must be validated.*

## Part 1: The Takeaways

- ✓ An **LLM** is a scaled-up decoder transformer that **predicts the next word**
- ✓ **Scale** (data, parameters, compute) is what unlocked their surprising abilities
- ✓ They are **general-purpose text tools** — summarize, translate, classify, and more
- ✓ You steer them by **prompting** (zero-shot, few-shot, chain-of-thought)
- ✓ For research, **data annotation** is the killer app — but **always validate** against human coders

**Next (Part 2):** the harder questions — ethics, responsible use, why **open & local** models matter, and **distillation** to get the best of both worlds.